



카테고리 없음

[LoRA] 논문 리뷰

yuha933 2025. 7. 8. 16:23

1. Introduction

• 배경

- 현대 NLP 응용은 하나의 대규모 사전학습(pre-trained) 언어모델을 다양한 다운스트림 과제에 맞춰 fine-tuning하는 방식을 채택합니다.
- 하지만 fine-tuning은 모델의 모든 파라미터를 업데이트하므로, 과제별로 모델 전체를 저장·배포해야 해 비용 및 관리 측면에서 비효율적이며, GPT-3(175B)와 같은 극대화된 모델에서는 현실적인 장애가 됩니다.
- 그니까 정리하면,
 - 범용 텍스트로 미리 커다란 모델 학습 -> 각 모델에 맞춰 추가 학습할 때, 원래 학습된 모델의 모든 파라미터를 그 도메인 데이터에 맞춰 파인튜닝 진행
 - 여기서 문제 ! 모델 크기가 수십억 ~ 수백억 파라미터이다보니, 파인튜닝하다 모델 전체를 새로 저장하고 관리하는 데에 문제가 생김

• 기존 파라미터 효율적 적응 기법의 한계

- **Adapter 삽입**: 과제별 작은 모듈만 학습하지만, 네트워크 깊이를 늘려 추론 시 추가 지연(latency)이 발생합니다.
- **Prompt/Prefix 튜닝**: 소수의 토큰만 학습하지만, 시퀀스 길이가 줄어들어 표현력이 제한되고,
- **기타 저파라미터 학습 기법**: full fine-tuning 성능에 근접하지 못하는 경우가 많아 “효율”과 “품질” 간 트레이드오프가 존재합니다.

• LoRA 제안 동기

- 선행 연구(예: Li et al. 2018; Aghajanyan et al. 2020)는 과대파라미터화된 모델이 실제로는 낮은 차원(intrinsic dimension) 위에서 동작함을 보여주었습니다.
- 본 논문은 “적응(adaptation) 중 모델 가중치 변화분(ΔW) 역시 저(低)랭크 구조를 가질 것”이라 가정하고, 이를 효율적으로 학습하는 **Low-Rank Adaptation (LoRA)** 기법을 제안합니다.

• LoRA 핵심 아이디어

- 사전학습된 가중치 W_0 는 고정(frozen)하고, 모델 업데이트 ΔW 를 두 개의 저랭크 행렬 $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ 의 곱으로 표현: $\Delta W = BA$, $r \ll \min(d, k)$
- 초기화: $B=0$, $A \sim \mathcal{N}(0, \sigma^2)$ 로 설정해 학습 초기에 $\Delta W=0$ 이 되도록 함.
- 순전파 시: $h = W_0 x + \Delta W x = W_0 x + B(Ax)$, $x = W_0^{-1} h$, $h = W_0 x + \Delta W x$, $x = W_0^{-1} h$, $(Ax)h = W_0 x + \Delta W x = W_0 x + B(Ax)$
- 과제 전환 시 A, B 만 교체하면 되어 저장·로드 비용이 극히 작고, 배포 시 $W_0 + BA$ 로 병합(merge)해 **추론 지연 없이** 사용할 수 있습니다.

• LoRA의 주요 장점

- **극소수 파라미터 학습**: 전체 파라미터의 0.01%~1%만 최적화
- **저장 공간 절감**: 과제별 모듈(A, B)만 저장 → 디스크 사용량·전송 비용 감소
- **추론 효율 유지**: 학습된 저랭크 행렬을 사전학습 가중치에 병합하여 추가 지연 없이 사용
- **학습 자원 절감**: adaptive optimizer 사용 시 최적화해야 할 파라미터가 작아 GPU 메모리·연산량 최대 3배 절감

- 기타 기법과의 병행성: prefix-tuning 등 여타 parameter-efficient 방법과 결합 가능

• 용어 및 구성

- Transformer 층 차원 d_{model}
- 자기어텐션 투영 행렬: W_q, W_k, W_v, W_o
- ΔW : 적응 중 누적된 가중치 변화
- rank r : LoRA 모듈의 저랭크 차원
- 옵티마이저: Adam 등

2. Problem Statement

- 조건부 언어 모델링 (문맥 x 가 주어졌을 때, 정답 y 를 생성하는 조건부 확률 $P(y|x)$)을 최대화하는 문제

• Full Fine-Tuning 방식

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log P_{\Theta}(y_t | x, y_{<t})$$

- 학습해야 할 파라미터 수 = $d \times k$
($W_0 \in \mathbb{R}^{d \times k}$)

단점

기존 파인튜닝은 바꿔야 할 파라미터 수가 모델 크기 전체와 같아서, 과제별로 전체 모델을 저장·관리해야 해 메모리 비용 부담이 매우 큼

• Parameter-Efficient Formulation

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log P_{\Phi_0 + \Delta\Phi(\Theta)}(y_t | x, y_{<t})$$

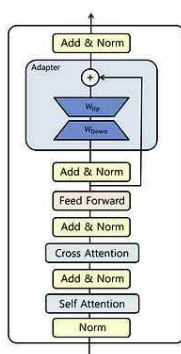
- 학습해야 할 파라미터 수 = $r(d + k)$
($\Delta W = B \cdot A, B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}$)

아이디어

과제 특화 파라미터 ΔW 을 더 작은 파라미터 집합 Θ 로 인코딩

3. Aren't Existing Solutions Good Enough?

• Adapter Layers 문제



• 전체 흐름

1. 원본 모델 순전파
2. Adapter 모듈
3. 최종 출력

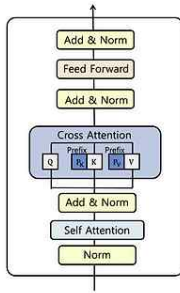
• Adapter 내부 흐름

1. h 차원 축소
2. ReLU
3. 차원 복원
4. Δh 을 원래 h 에 더함

- 장점: 학습해야 할 파라미터가 전체의 1% 미만으로 줄어듦

- 문제: 추론 시 Adapter 연산이 순차적으로 추가되어, 한 층당 연산이 하나 더 늘어나면서 지연 발생

• Prefix-Tuning 문제



• Prefix-Tuning 흐름

1. 원본 토큰만으로 Self-Attention
2. key와 value 앞에 Prefix 토큰을 붙여 Cross Attention
(Prefix 토큰이 사전 정보 역할을 하며 어텐션 계산)
3. Feed-Forward, Add&Norm 연산 수행
4. 최종 출력

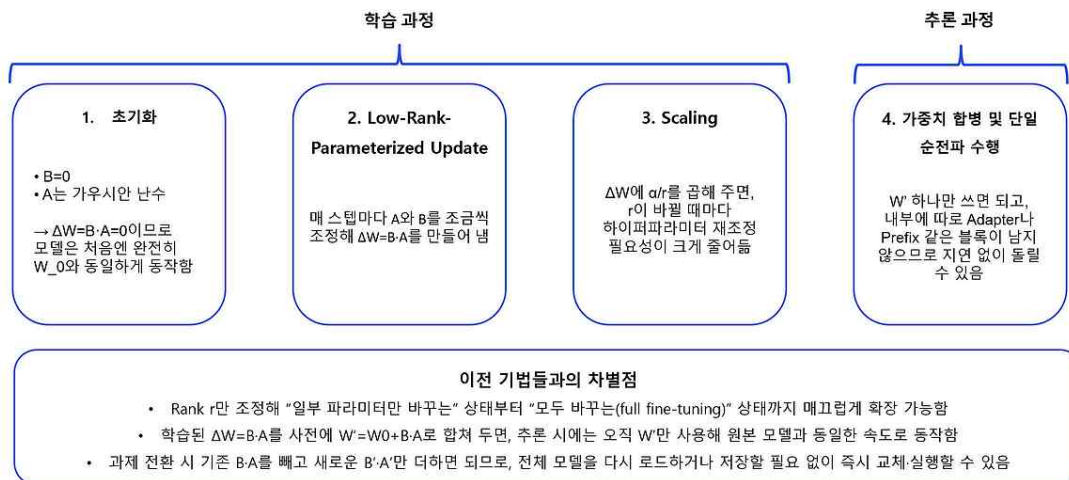
• 장점 : 가상 토큰 임베딩만 학습시킴으로써 매우 적은 파라미터로 모델을 적응시킬 수 있고, 모델 구조 변경 불필요

• 문제

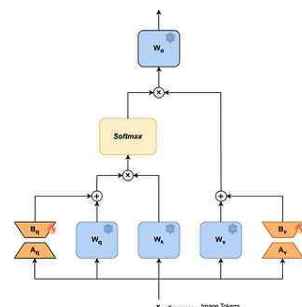
- Prefix 길이만큼 원본 입력 길이가 줄어들
- 어텐션 계산량이 $(n + l)^2$ 로 증가하기 때문에 추론 지연이 발생함
- 길이 초기화 민감도로 성능 안정화 어려움

4. Our Method

• Low-Rank-Parameterized Update Matrices



• Applying LoRA To Transformer



• Transformer에서의 가중치

- Self-Attention에 4가지 투영 행렬(W_q, W_k, W_v, W_o)
- MLP(피드포워드)에 2개의 가중치 행렬

• Query, Value에만 적용한 이유

- 경험적 선택 (효과가 검증된 최소한의 부분)
- 둘의 조정만으로 모델의 동작 방식을 거의 완벽히 튜닝할 수 있음
 - Q : 어떤 정보에 집중할지 → Attention 분포 직접 조정
 - V : 어떤 정보를 가져올지 → Attention 결과 컨트롤

한계

- 한 배치에 서로 다른 과제(task)가 섞여 있을 때 그 배치에 맞춰 매 샘플마다 다른 B·A를 반영하는 게 어려움
 - 다른 레이어(MLP, LayerNorm, bias)에 적용은 미검증으로 남아있음

5. Empirical Experiments

Experimental Setup

모델

- NLU: RoBERTa (base/large), DeBERTa-XXL
- NLG: GPT-2 (medium/large), GPT-3 175B

과제

- 분류 (GLUE)
 - MNLI, SST-2, MRPC, CoLA, QNLI, QQP, RTE, STS-B
- SQL 생성
 - WikiSQL (자연어 → SQL)
- 대화 요약
 - SAMSum
- 텍스트 생성
 - E2E NLG Challenge

비교 기법

- Full fine-tune (FT)
 - : 파인튜닝 시 모델은 사전학습된 가중치와 편향으로 초기화된 뒤 모든 파라미터가 그래디언트 업데이트를 거치는 방식
- Bias-only (BitFit)
 - : 모든 파라미터를 고정하고 편향(bias) 벡터만 학습하는 방식
- Prefix-embedding/Prefix-layer Tuning
 - : 입력 토큰 사이에 학습 가능한 가상 토큰을 삽입하여, 이 토큰들의 워드 임베딩만 학습하는 방식
- Adapter variants (Houlsby-style, Lin-style)
 - : Self-Attention과 MLP 모듈 사이에 작은 MLP 형태의 어댑터를 삽입하고, 해당 어댑터 블록만 학습하는 방식
- LoRA
 - : 기존 가중치 행렬 옆에 저랭크 분해 행렬 쌍을 병렬로 추가 학습하는 방식

RoBERTa & DeBERTa-XXL

Model & Method	# Trainable Parameters	E2E NLG Challenge									
		MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.	
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4	
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2	
RoB _{base} (Adpt ^L)*	0.3M	87.1 $\pm$ 0	94.2 $\pm$ 1	88.5 $\pm$ 1.1	60.8 $\pm$ 4	93.1 $\pm$ 1	90.2 $\pm$ 0	71.5 $\pm$ 2.7	89.7 $\pm$ 3	84.4	
RoB _{base} (Adpt ^H)*	0.9M	87.3 $\pm$ 1	94.7 $\pm$ 3	88.4 $\pm$ 1	62.6 $\pm$ 9	93.0 $\pm$ 2	90.6 $\pm$ 0	75.9 $\pm$ 2.2	90.3 $\pm$ 1	85.4	
RoB _{base} (LoRA)	0.3M	87.5 $\pm$ 3	95.1 $\pm$ 2	89.7 $\pm$ 7	63.4 $\pm$ 1.2	93.3 $\pm$ 3	90.8 $\pm$ 1	86.6 $\pm$ 7	91.5 $\pm$ 2	87.2	
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9	
RoB _{large} (LoRA)	0.8M	90.6 $\pm$ 2	96.2 $\pm$ 5	90.9 $\pm$ 1.2	68.2 $\pm$ 1.9	94.9 $\pm$ 3	91.6 $\pm$ 1	87.4 $\pm$ 2.5	92.6 $\pm$ 2	89.0	
RoB _{large} (Adpt ^L)*	3.0M	90.2 $\pm$ 3	96.1 $\pm$ 3	90.2 $\pm$ 7	68.3 $\pm$ 1.0	94.8 $\pm$ 2	91.9 $\pm$ 1	83.8 $\pm$ 2.9	92.1 $\pm$ 7	88.4	
RoB _{large} (Adpt ^H)*	0.8M	90.5 $\pm$ 3	96.6 $\pm$ 2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8 $\pm$ 3	91.7 $\pm$ 2	80.1 $\pm$ 2.9	91.9 $\pm$ 4	87.9	
RoB _{large} (Adpt ^H)*	6.0M	89.9 $\pm$ 5	96.2 $\pm$ 3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$ 2	92.1 $\pm$ 1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8	
RoB _{large} (Adpt ^H)*	0.8M	90.3 $\pm$ 3	96.3 $\pm$ 5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$ 2	91.5 $\pm$ 1	72.9 $\pm$ 2.9	91.5 $\pm$ 5	86.4	
RoB _{large} (LoRA)*	0.8M	90.6 $\pm$ 2	96.2 $\pm$ 5	90.2 $\pm$ 1.0	68.2 $\pm$ 1.9	94.8 $\pm$ 3	91.6 $\pm$ 2	85.2 $\pm$ 1.1	92.3 $\pm$ 5	88.6	
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1	
DeB _{XXL} (LoRA)	4.7M	91.9 $\pm$ 2	96.9 $\pm$ 2	92.6 $\pm$ 6	72.4 $\pm$ 1.1	96.0 $\pm$ 1	92.9 $\pm$ 1	94.9 $\pm$ 4	93.0 $\pm$ 2	91.3	

*RoBERTa-base:

- FT : 86.4% → LoRA : 87.2%

* RoBERTa-large:

- FT : 88.3% → LoRA : 89.0%

* DeBERTa-XXL:

- FT : 91.1% → LoRA : 91.3%

* BitFit, Prefix, Adapter 모두 FT 대비 성능 격차 존재

GPT-2

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^H)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)*	11.09M	67.3 $\pm$ 6	8.50 $\pm$.07	46.0 $\pm$ 2	70.7 $\pm$ 2	2.44 $\pm$.01
GPT-2 M (FT ^{top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4 $\pm$ 1	8.85 $\pm$.02	46.8 $\pm$ 2	71.8 $\pm$ 1	2.53 $\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)*	0.88M	69.1 $\pm$ 1	8.68 $\pm$.03	46.3 $\pm$ 0	71.4 $\pm$ 2	2.49 $\pm$.0
GPT-2 L (Adapter ^L)*	23.00M	68.9 $\pm$ 3	8.70 $\pm$.04	46.1 $\pm$ 1	71.3 $\pm$ 2	2.45 $\pm$.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4 $\pm$ 1	8.89 $\pm$.02	46.8 $\pm$ 2	72.0 $\pm$ 2	2.47 $\pm$.02

* GPT-2 medium:

- FT(355M): NIST=8.62, ROUGE-L=71.0
→ LoRA(0.35M): NIST=8.85, ROUGE-L=71.8

* GPT-2 large:

- FT(774M): NIST=8.78, ROUGE-L=69.9
→ LoRA(0.77M): NIST=8.89, ROUGE-L=72.0

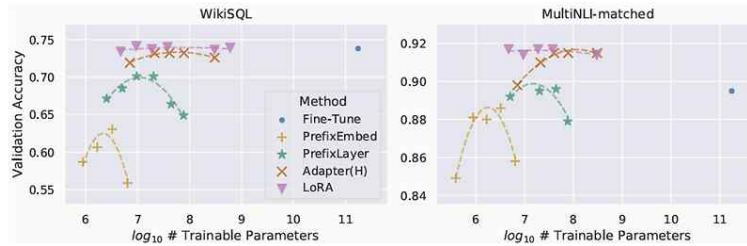
* LoRA는 텍스트 생성 품질을 adapter나 prefix-tuning보다 더 잘 유지함

GPT-3 175GB

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

- WikiSQL
 - FT(175B) 73.8% → LoRA(4.7M) 73.4%, LoRA(37.7M) 74.0%
- MultiNLI-matched
 - FT 89.5% → LoRA 91.6%
- SAMSum: R1/R2/RL
 - FT 52.0/28.0/44.5 → LoRA 53.4/29.2/45.1

Scalability and Task-Performance



- x축 : 학습 가능한 파라미터 수와 로그 스케일
- y축 : 검증 정확도
- LoRA는 매우 적은 추가 파라미터로도 전체 파인튜닝 수준의 성능을 달성하며, 파라미터-성능 곡선이 다른 기법들보다 훨씬 안정적임

6. Understanding The Low-Rank Updates

- 주어진 파라미터 예산 하에서, 사전학습된 Transformer의 어떤 가중치 행렬 부분을 어댑트하는 것이 다운스트림 성능을 최대로 이끌어낼까?

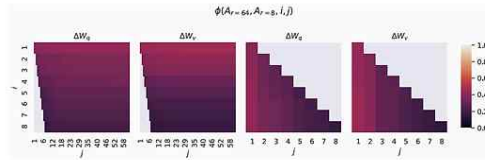
	# of Trainable Parameters = 18M							
Weight Type	W_q	W_k	W_v	W_o	W_q, W_k	W_q, W_v	W_q, W_k, W_v, W_o	
Rank r	8	8	8	8	4	4	2	
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7		73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3		91.7

- 가중치 한 종류($r=8$)
 - ΔW 의 표현 공간은 충분하지만, 하나의 가중치 행렬에만 모든 능력을 몰아줘서 정보가 편중됨
- 가중치 네 종류($r=2$)
 - 네 군데에 고루 퍼뜨리긴 하지만, 각 ΔW 가 너무 가늘어져서 세부 조정 능력이 부족해짐
- 가중치 두 종류($r=4$)
 - W_{query} (쿼리)와 W_{value} (값) 두 군데만 중간 정도 굵기로($r=4$) 퍼뜨려 줘서, ΔW 가 담을 수 있는 정보량이 충분하면서 파라미터를 골고루 분산해 어텐션 조정 성능을 극대화하는 최적의 균형을 이룸

- 최적의 어댑테이션 매트릭스 ΔW 는 정말 저랭크인가? 그렇다면 실제로 사용하기 좋은 랭크는 무엇일까?

	Weight Type	$r=1$	$r=2$	$r=4$	$r=8$	$r=64$
WikiSQL ($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_o, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
	W_q	90.7	90.9	91.1	90.7	90.7
MultiNLI ($\pm 0.1\%$)	W_q	91.3	91.4	91.3	91.6	91.4
	W_o, W_v	91.2	91.7	91.7	91.5	91.4
	W_q, W_k, W_v, W_o					

- 4개 행렬 모두에 LoRA를 붙이는 경우
 - 파라미터 한도(18M)를 지키기 위해 랭크를 낮춰야 해서 WikiSQL 정확도는 74.1 → 73.7로 0.4포인트 떨어짐
- 쿼리-값 두 개에만 붙이는 경우
 - 랭크 4 하나로 양쪽 과제 모두 거의 최댓값(73.7 / 91.3)을 달성함
- W_q, W_v 만 튜닝하는 것이 예산 내에서 성능을 최적화하는 가장 효율적인 선택

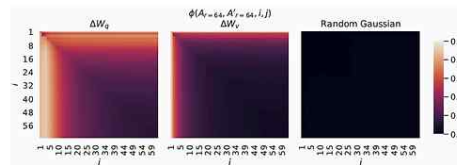


- X축 j : Ar=64에서 뽑은 상위 j개 방향
- Y축 i : Ar=8에서 뽑은 상위 i개 방향
- 색상 : 각각의 i차원과 j차원이 얼마나 겹치는 지를 정규화한 값
- 왼쪽 두 패널 : rank 8인 쪽의 상위 i축과 rank 64인 쪽의 상위 j축이 얼마나 겹치는지
- 오른쪽 두 패널 : rank 8 쪽의 상위 8개 축만 골라, rank 64 쪽의 상위 1~8개 축과 비교
- LoRA가 실제로 학습하는 본질적인 차원은 매우 작아 1~2 수준에 불과함을 보임

- ΔW와 원본 가중치 W 사이에는 어떤 연관성이 있을까? ΔW는 W에 비해 얼마나 큰가?

	$r = 4$			$r = 64$		
	ΔW_q	W_q	Random	ΔW_q	W_q	Random
$\ U^T W_q V^T\ _F =$	0.32	21.67	0.02	1.90	37.71	0.33
$\ W_q\ _F = 61.95$	$\ \Delta W_q\ _F = 6.91$			$\ \Delta W_q\ _F = 3.57$		

- ΔW와 원본 가중치 W의 상관관계
 - ΔW의 학습된 업데이트 방향들은 원본 가중치의 주요 방향보다 훨씬 작지만 무작위 행렬보다는 훨씬 크게 겹침
 - 원본 가중치에 있던 일부 방향을 골라 증폭하고 있음을 보임
- 증폭 정도
 - 큰 증폭 비율을 보임
 - ΔW가 학습 과정에서, 원래 가중치에서 덜 강조되었던 축을 크게 부각시키고 있음을 나타냄



- i : Rank 8 쪽에서 중요한 j개 축
- j : Rank 64 쪽에서 가장 중요한 j개 축
- 색상 : 진할수록 상위 i축과 상위 j축이 많이 겹침
- LoRA가 실제로 '학습하는' 중요 축은 1개 정도로 매우 낮고, 나머지는 모두 노이즈(무작위)처럼 달라짐

7. Conclusion And Future Work

• LoRA의 의의

- 추론 지연을 추가하지 않음
- 입력 시퀀스의 길이를 줄이지 않음
- 모델 품질을 높게 유지함
- 신속한 과제 전환을 가능하게 함
- 밀집층이 있는 모든 신경망에 적용할 수 있음

• 향후 연구 방향

- 다른 효율적 적응 기법들과 결합해 성능 향상
- 파인튜닝 메커니즘 규명
- 경험적인 아닌 원리적인 LoRA 적용 대상 선택법 연구
- 원본 가중치 W 자체의 저랭크 가능성 탐구